

Université de Ngaoundéré

Master 2

Data Mining, Fouille de données
et Statistiques décisionnelles

Chapitre 1

Fondamentaux de la fouille de données et prétraitement

Corrigés

Dispensé par

Pr Ndam Njoya Arouna

Chapitre 1 — Corrigés

0.1 Exercice 1.1

Script. `correction_ex01_audit.py` génère le CSV et affiche un audit de référence.

Attendu (ordre de grandeur).

- $n = 150$, $p = 4$;
- Manquants sur `montant_mensuel` : environ 8 % (`df.isna().mean() * 100`);
- 1 **doublon** sur (`nom`, `email`) → en pratique, conserver une ligne après vérification métier ou fusionner les montants selon la règle documentée;
- IQR : `q1`, `q3` = `col.quantile(0.25)`, `col.quantile(0.75)`; `iqr` = `q3 - q1`; bornes `q1 - 1.5*iqr`, `q3 + 1.5*iqr`; le script affiche le nombre de flags (souvent 1 à 3 selon l'échantillon aléatoire).

Commentaire type. Le taux de manquants modéré impose de **documenter** la cause (saisie, refus) avant d'imputer. Les outliers IQR ne sont pas forcément des erreurs : validation métier avant exclusion.

—

0.2 Exercice 1.2

Dominance du CA. Avec $A = (30 \text{ ans}, 100 \text{ k€})$ et $B = (35 \text{ ans}, 5\,000 \text{ k€})$, $\Delta x_1 = 5$, $\Delta x_2 = 4900 \rightarrow$ contribution au carré $\approx 25 + 24\,010\,000$, quasi entièrement due au CA.

Après z-score (sur un échantillon de référence), les écarts sont ramenés à des unités comparables; la distance reflète mieux les deux dimensions.

Script. `correction_ex02_distance.py` illustre le calcul numérique.

—

0.3 Exercice 1.3

Structure type.

```
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

num_cols = ["montant_mensuel"]
pre = ColumnTransformer([
    ("num", Pipeline([
        ("imp", SimpleImputer(strategy="median")),
        ("sc", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("imp", SimpleImputer(strategy="most_frequent")),
```

```

        ("oh", OneHotEncoder(handle_unknown="ignore")),
    ]), ["ville"]),
])
pre.fit(X_train)
X_train_t = pre.transform(X_train)
X_test_t = pre.transform(X_test)

```

Forme. $p = 1 + (\text{nombre de modalités de ville après encodage})$ — typiquement **4 colonnes** (1 standardisée + 3 villes si 3 modalités vues au train).

Fuite. Médiane (ou moyenne / std) calculée sur train+test **incorpore** des informations du test dans le prétraitement → scores optimistes et invalides en généralisation.

Production. Une nouvelle modalité de **ville** n'a pas été vue au **fit** ; **ignore** évite le plantage et met les colonnes à zéro plutôt qu'une erreur.

0.4 Exercice 1.4

Recommandation type. Analyser le mécanisme de manque (tableaux croisés manquant × secteur), imputer par segment avec **indicateur** « revenu imputé », tester la sensibilité du clustering ou du score final.

0.5 Exercice 1.5

1. **Sélection** → **Prétraitement** → **Transformation** → **Fouille** → **Interprétation**. La fouille = étape 4.
2. **Faux.** Sans nettoyage, les motifs extraits sont fragiles ou trompeurs.
3. Les villes n'ont **pas d'ordre** naturel ; des entiers imposeraient un ordre artificiel et fausseraient distances / coefficients.
4. Exemples : MCAR = panne aléatoire de capteur ; MAR = les jeunes omettent plus souvent le revenu ; MNAR = les très hauts revenus laissent la case vide.
5. **'fit'** apprend les paramètres (médianes, catégories, etc.) sur les données d'apprentissage ; **'transform'** applique ces paramètres fixes à d'autres jeux.
6. Exemples acceptés : histogrammes / boxplots par variable, matrice de corrélations, barres des manquants, comptages de modalités.